

GQH Hardware Track Participant Guide

This guide covers everything your team needs for the GQH Hardware Track: borrowing and returning your FPGA, setting up and programming the Tang Nano 20K, how judging works, and submitting your final project.

Minimum viable submission

Receive 8 bytes → decode items A and B → maintain two independent 16-price windows → compute crossings → send exactly 8 bytes back → pass `21_quick_uart_test.py` → pass `22_robust_uart_test.py`.

During the official run, no team-supplied host software is executed. All UART parsing, state, computation, and response generation happen on the FPGA.

Part 1: FPGA Pickup & Drop-off Procedures

Overview

Each registered hackathon team borrows one FPGA from the Internet of Things Club at pickup on Friday, October 2, and returns it at drop-off by Sunday, October 4. Judges then program and test every board. Every board is checked out and checked back in individually so the club can track all of its equipment.

	Pickup	Drop-off
Date & time	Friday, October 2, 7:15 pm	Sunday, October 4, by 11:00 am
Location	Reitz Room 2345	Reitz Room 2345
Requirement	Team registered and FPGA Loan Agreement signed	Final code submitted to GitHub and Devpost; FPGA handed to the judges

Pickup

FPGAs are handed out at Reitz Room 2345 at 7:15 pm on Friday, October 2nd, one per team, only after your team has registered and every member has signed the FPGA Loan Agreement on the sign-up form.

Priority by team size. If there are more teams than boards, FPGAs are assigned by team size:

1. Teams of 4
2. Teams of 3
3. Teams of 2
4. Individuals

Within the same team size, boards go out in the order teams completed sign-up. Teams that do not receive a board will be notified by email before the event.

What to bring.

- At least one team member (ideally the member the board is checked out under)
- A student ID for identity verification

- A bag or case to carry the board safely

Check-out steps.

1. Give the team name and show a student ID to the officer at the table.
2. The officer confirms the team is registered and every member has signed the loan agreement.
3. The officer records the board's serial number or asset tag next to the team name.
4. Inspect the board and accessories with the officer, and report any existing damage or missing items before leaving.
5. Sign the check-out sheet to confirm you received the board in working condition.

Drop-off

Return the FPGA and all accessories to Reitz Room 2345 by Sunday, October 4 at 11:00 am, after your team has submitted its final code.

Before you come.

- Push your final code to your team's GitHub repository and complete your Devpost entry with the repository link and full commit SHA.
- Make sure your final .fs file is in the repository. Judges program your board from it.
- Power down and disconnect the board.
- Gather every accessory you received: cables, power supply, and the box or bag.

Check-in steps.

1. Give the team name to the officer at the table.
2. The officer confirms your GitHub submission was received.
3. The officer matches the board's serial number or asset tag to your team's check-out record.
4. Inspect the board and accessories with the officer for damage or missing items.
5. Sign the check-in sheet. Your team's loan is closed once the officer marks the board as returned.

After drop-off: judging

Judges program each board in SRAM mode with the .fs file from your submitted commit and test it on a single judging PC using the official test. You do not need to keep the board powered or connected. Your submission must be final before drop-off: changes pushed afterward are not judged. If a run fails for a reason outside your design (cable, wrong COM port), judges may reprogram and rerun once.

Late, lost, or damaged boards

The terms in the FPGA Loan Agreement apply to every board, including late returns, damage, and replacement costs.

- **Can't make drop-off:** contact an organizer before the deadline to arrange another time. Another team member may return the board for you.
- **Damaged board:** report it at drop-off rather than hiding it. The officer will note the damage on the check-in sheet.
- **Lost or stolen board:** notify an organizer as soon as you notice it is missing.

Questions

Contact the hackathon organizers on our Discord (discord.gg/9qPMtN4UB) or email us at IoTStudentsClub@ece.ufl.edu with any questions about pickup or drop-off.

Part 2: FPGA Setup & Usage

1. Installation of Gowin EDA

1. Go to the Gowin Semiconductor download page for "GOWIN EDA" and create your account.
2. Select the option **V1.9.11.03 Education**. On Windows, choose "Windows x64". On Linux, use the Linux tab, decompress the ZIP file, and run `gw_ide` from the `/IDE/BIN` folder. The download page lists Windows and Linux builds only.
3. Connect the board with a data-capable USB-C cable. It should appear as "USBTAG/serial device".

2. Creating a project in Gowin EDA

1. Go to File → New → FPGA Design Project. Choose a name and folder.
2. Select the device using the settings below.
3. Add your source files with File → New (for example a VHDL File), or right-click the project and add existing files. Keep the correct extension (`.vhd` or `.v`).
4. Go to Project → Configuration → Synthesize → General → Top Module/Entity and enter the name of **your design's actual top-level entity/module**.
 - The name of your top-level entity is up to you. What matters is that the correct entity/module is selected as the Gowin synthesis root, the top-level port names match the organizer-supplied `.cst` file, and the design obeys the UART and packet interface in section 10.

Parameter	Selection
Series	GW2AR
Package	QFN 88
Device	GW2AR-18
Device Version	C
Part number	GW2AR-LV18QN88C8/I7

3. Writing code

This section is a warm-up only. Its examples use port names (`clk`, `btn`, `rst0`, `rst4`, `led`) that are not in the competition `.cst`, so they will not run on the board as written. Your competition design must use the port names in section 9.

VHDL design files have three parts: libraries, entities (chip pins from outside), and the architecture (what happens inside the chip).

```
library ieee;
use ieee.std_logic_1164.all; -- gives the logic types
```

```

use ieee.numeric_std.all;      -- math on vectors

entity top is
  -- generic list: constants that configure the block but are not physical pins.
  generic (
    CLK_FREQ : natural := 27_000_000 -- constants are UPPERCASE; natural = integer
  );

  -- list of ports: the actual signals entering and leaving the block
  -- in:  signal enters the block, you can read but not assign to it
  -- out: the signal is assigned
  port (
    clk : in  std_logic;           -- std_logic is a single bit
    btn : in  std_logic;
    led : out std_logic_vector(5 downto 0) -- 5 downto 0 gives 6 bits
  );
end entity;

architecture rtl of top is
  -- DECLARATIVE PART: internal signals, constants, types
  constant HALF_SEC : natural := 13_500_000;
  signal cnt         : natural range 0 to HALF_SEC - 1 := 0;
  signal pattern      : unsigned(5 downto 0) := "000001";
begin
  -- STATEMENT PART: everything here runs in parallel

  -- a) Sequential logic (process with clock)
  process(clk)
  begin
    if rising_edge(clk) then -- rising edge: clk goes from 0 to 1
      null;                  -- clocked logic goes here
    end if;
  end process;

  -- b) Concurrent assignment (pure wiring / combinational)
  -- a permanent wire which is always active.
  led <= not std_logic_vector(pattern);
end architecture;

```

In this example, the architecture uses both sequential logic and concurrent assignment. Use sequential logic if the output depends on stored values and inputs. Use concurrent assignment (combinational logic) if the output depends only on the current inputs, such as gates, muxes, decoders, and adders.

Example: odd counter

Design a counter that follows the sequence $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$. It advances every 0.5 s and shows the count on LEDs 0–2.

Step 1: Design the table of decimal and binary representation.

Decimal	Binary (Q2 Q1 Q0)
1	001
3	011

5	101
7	111

Step 2: Write the code. The odd sequence occurs when Q0 is 1. Therefore, with cnt (0) held at 1, the counter always shows odd values, while the pair Q2 Q1 alternates its values.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity top is
  port (
    clk : in std_logic;           -- 27 MHz
    rst0 : in std_logic;          -- S1: async reset to "000"
    rst4 : in std_logic;          -- S2: async reset to "100"
    led : out std_logic_vector(5 downto 0) -- active LOW on the board
  );
end entity;

architecture rtl of top is
  constant HALF_SEC : natural := 13_500_000;
  signal div : natural range 0 to HALF_SEC - 1 := 0;
  signal tick : std_logic;
  signal cnt : std_logic_vector(2 downto 0) := "001";
begin
  -- Combinational: one-clock pulse every 0.5 s
  tick <= '1' when div = HALF_SEC - 1 else '0';

  -- Sequential: clock divider
  process(clk)
  begin
    if rising_edge(clk) then
      if tick = '1' then
        div <= 0;
      else
        div <= div + 1;
      end if;
    end if;
  end process;

  -- Sequential: counter with two ASYNCHRONOUS resets
  process(clk, rst0, rst4) -- resets in the sensitivity list!
  begin
    if rst0 = '1' then      -- checked BEFORE the clock edge
      cnt <= "000";
    elsif rst4 = '1' then
      cnt <= "100";
    elsif rising_edge(clk) then
      if tick = '1' then
        cnt(0) <= '1';      -- Q0+ = 1
        cnt(1) <= not cnt(1); -- Q1+ = Q1'
        cnt(2) <= cnt(2) xor cnt(1); -- Q2+ = Q2 xor Q1
      end if;
    end if;
  end process;
end architecture;

```

```

    end if;
end process;

-- Board LEDs are active LOW, so invert. LEDs 3-5 stay off.
led <= "111" & not cnt;
end architecture;

```

4. Pin assignment

1. The Hardware Track organizers provide the Tang Nano 20K physical constraint file: `19_tang_nano_20k.cst`.
2. Participants are not expected to create their own `.cst` file or determine FPGA pin assignments.
3. Add the supplied `.cst` to your Gowin project and use it as the project's physical constraint file: open your project, select the Design tab on the left, right-click, and choose Add Files.
4. The competition mappings are listed in section 9. Your top-level port names must match them.
5. Do not manually recreate the competition pin assignments in FloorPlanner.

5. Program the board

First run Synthesize and then Place & Route. Both must finish without errors. The bitstream is written to `impl/pnr/<project>.fs`.

1. With the Tang Nano 20K plugged into your computer, click the Programmer icon or go to Tools → Programmer.
2. Click Scan Device and select the GW2AR-18C row.
3. Double-click the device row to open Device Configuration and use the settings below.
4. Click Save.
5. Click Program/Configure and check the console status.

Parameter	Setting
Access Mode	SRAM Mode
Operation	SRAM Program
File name	Browse to <code>impl/pnr/<project>.fs</code>

Judging uses SRAM programming, so you do not need to write to external flash. SRAM contents are lost when power is removed.

6. Simulation and programming fallback

Simulation is recommended but not required. Gowin EDA does not simulate designs, so use a separate simulator with your own testbench to check your logic before you synthesize. Acceptable options:

1. A university- or company-provided licensed Siemens Questa installation.
2. Questa-Intel FPGA Starter Edition: see the Intel installation and licensing pages and the Questa-Intel FPGA quick start.
3. GHDL with GTKWave, a free open-source option that the organizers do not support.

Do not add a simulation-only testbench to the synthesis sources.

Optional programming fallback: MSYS2 + openFPGALoader. Use it only if the Gowin Programmer cannot program the board. Install MSYS2 from msys2.org. Open MSYS2 UCRT64, not the normal Command Prompt. Then update the package database and install core packages:

```
pacman -Syu
```

If MSYS2 asks you to close and reopen the terminal, reopen UCRT64 and run `pacman -Syu` again. Install openFPGALoader (the official UCRT64 package) and confirm it is available:

```
pacman -S mingw-w64-ucrt-x86_64-openFPGALoader
openFPGALoader --Version
```

Confirm the Tang Nano 20K board definition exists, then program a Gowin .fs file into volatile FPGA SRAM:

```
openFPGALoader --list-boards | grep tangnano20k
# -b selects the board; omitting -f means volatile SRAM programming.
openFPGALoader -b tangnano20k "/c/path/to/project.fs"
```

Do not add `-f` unless you intentionally want persistent external-flash programming.

7. Files provided by the organizers

File	Purpose
19_tang_nano_20k.cst	Board pin mappings. Add it to your Gowin project.
21_quick_uart_test.py	Quick sanity test: basic communication and packet format. Run this first.
22_robust_uart_test.py	Scoring-style test with a software reference model. Run this second.

Both scripts need Python 3 and pyserial (`pip install pyserial`). Change only the `PORT` setting to your board's COM port. Do not change the packet protocol or the scoring logic to make a design pass. The robust test saves a CSV of every packet. Keep it: it is the best debugging tool you have. The .cst file is provided because board-specific pin discovery is not the intended algorithmic challenge. The UART modules, packet logic, and testbenches are yours to write.

8. Project structure

How you organize your HDL is up to you. Teams may combine modules, split modules, rename files, and organize their design however they prefer. A valid project only needs to:

- contain a synthesizable top-level entity/module, set as the Top Module/Entity in Gowin;
- use the external ports and names from the supplied .cst;
- implement the UART packet protocol in section 10; and
- reproduce the moving-average behavior in section 10.

If you want a starting point, a natural breakdown is a UART receiver and transmitter, an 8-byte packet receiver and transmitter, an item router, one moving-average engine per item (16-price window, rolling sum, crossing detector, held action), and an output packet builder.

9. Pin mappings

Top-level port names must match the `.cst` exactly. Do not rename any port: the names are fixed for this competition.

Port	FPGA pin	Direction	Purpose
sys_clk	4	in	On-board 27 MHz clock (can be swapped for different internal clock signals)
reset_btn	87	in	User reset button, pull-down (optional usage)
uart_rx_i	70	in	BL616 → FPGA (UART input)
uart_tx_o	69	out	FPGA → BL616 (UART output)
led0_n	15	out	Active-low status LED (optional usage)
led1_n	16	out	Active-low status LED (optional usage)

If you do not use an optional port, keep it in your top-level port list and leave it unconnected (drive an unused LED high).

10. Design guidelines

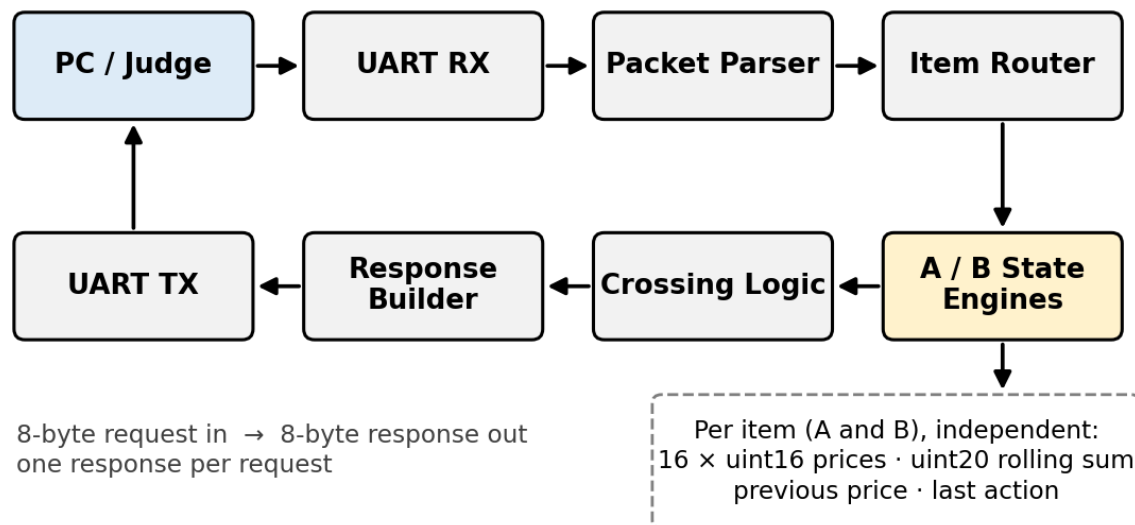
Your design must obey the external protocol exactly; how you organize the HDL inside is up to you. During the official run, no team-supplied host software is executed. All UART parsing, state, algorithmic computation, and response generation must occur on the FPGA.

Warning: the BL616 USB-serial bridge

The Tang Nano 20K's onboard BL616 can drop or corrupt bytes, and the judge can then log a TIMEOUT, if your design sends response bytes back-to-back with no idle time. A design whose logic is correct can still fail this way. Your design must compensate with enough idle time or buffering between response bytes.

The latency the judge measures includes any delay you add, so there is a trade-off between reliability and speed. Test with `22_robust_uart_test.py` and keep the CSV.

Overview of the data path. One request goes in, one response comes out:



10.1 Protocol you must meet

- **10.1.1 UART:** 115200 baud, 8 data bits, no parity, 1 stop bit; LSB first. Multi-byte fields are big-endian.
- **10.1.2 Request:** exactly 8 bytes, most-significant byte first: index (2), item1 (1), price1 (2), item2 (1), price2 (2). Prices are unsigned 16-bit.
- **10.1.3 Response:** 8 bytes, most-significant byte first: index (2), item1 (1), action1 (1), item2 (1), action2 (1), reserved (2) = 0x0000. Index and item fields echo the request.
- **10.1.4 Action codes:** NONE = 0x00, SELL = 0x01, BUY = 0x02. When no crossing occurs, send the last action for that item again. NONE is sent only before that item's first crossing.
- **10.1.5 Item IDs:** Item A = 0x11 and Item B = 0x22. They are fixed for the entire competition.
- **10.1.6 One response per request.** Never send unsolicited bytes and never send a response before all 8 request bytes have arrived.
- **10.1.7 Slot order:** the response mirrors the request. If the request has Item B in slot 1, the response has Item B and its action in slot 1.

10.2 How a judging run is conducted

- **10.2.1** Each team is judged on one official run of **100 packets**, indices 0–99. The judge sends one request and waits for its complete response before sending the next (stop-and-wait).
- **10.2.2** Index 0 starts a new session and clears all state, so the board is not reset or reprogrammed between runs.
- **10.2.3** Indices 0–15 are warm-up, which leaves **84 scored packets and 168 scored actions**.
- **10.2.4** The price seed is chosen by the organizers and is the same for every team. It is not published in advance, so do not hardcode price patterns.
- **10.2.5** Judges program the board in SRAM mode with the .fs file from your submitted commit.

- **10.2.6** The official test runs on a single judging PC, with Gowin Programmer and any serial terminals closed before the judge opens the COM port.
- **10.2.7** A packet whose 8 response bytes do not all arrive within 1 second is logged as TIMEOUT. It counts as incorrect and ends the run, so packets not received score zero.
- **10.2.8** Judges may reprogram and rerun once if a run fails for a reason outside the design (cable, wrong COM port).
- **10.2.9** Keep the CSV from every local run you do. It is your best debugging tool.

10.3 Exact 16-sample moving-average algorithm

Each item maintains completely independent state: a window of its last 16 prices, the sum of those 16 prices, the previous price, and the most recently generated action (NONE until its first crossing). The sum of 16 unsigned 16-bit prices needs 20 bits.

Warm-up. Indices 0–15 only fill the first window. For each warm-up packet, add the item's price to its window and sum, **and store that price as the item's previous price**, so that at index 16 the previous price is the price from index 15. No crossing is evaluated during warm-up. Respond to warm-up packets with NONE for both actions. They are not scored for correctness, but their prices still count toward the average, and their responses must still follow the protocol and arrive in time.

Update procedure (index 16 onward). Perform these steps independently for each item:

1. **Old average:** $\text{old_average} = \text{old_sum} \gg 4$. This is floor division by 16. The fraction is discarded, not rounded.
2. **Slide the window:** $\text{new_sum} = \text{old_sum} - \text{oldest_price} + \text{current_price}$.
3. **New average:** $\text{new_average} = \text{new_sum} \gg 4$.
4. **BUY** if $\text{previous_price} \leq \text{old_average}$ AND $\text{current_price} > \text{new_average}$.
5. **SELL** if $\text{previous_price} \geq \text{old_average}$ AND $\text{current_price} < \text{new_average}$.
6. Otherwise no crossing has happened: repeat the item's last action. Store the current price as the new previous price.

The previous price is compared against the old average, and the newly received price against the new average. The new price is already included in the new average when the second half of the test is performed.

Reset. Index 0 starts a new session. On receipt of index 0, clear all previous session state first (every window, sum, previous price, and last action), then process index 0's two prices as the first samples of the new window. Your design must start a fresh session on index 0 by itself.

Item routing. Each item's window, average, and last action belong to the item ID, not to the packet slot. The tester may place either item in either slot on any packet. Route exclusively using the item ID, never by packet index or slot position, and respond in the same slot order as the request.

Previous price	Current price	Action
at or below the old average	above the new average	BUY: upward crossing
at or above the old average	below the new average	SELL: downward crossing
any other case		Repeat the last action; no crossing

Example: Item B, the average is 100.

Before	Now	Action	Comments
95	105	BUY	Crossed up
105	94	SELL	Crossed down
105	120	Repeat last action	No crossing

10.4 Worked example

64-bit request (judge to FPGA): bytes 00 10 11 00 50 22 00 C8

Bytes	Field	Hex	Meaning
0-1	index	00 10	$0x0010 = 1 \times 16 + 0 = 16$
2	item1	11	Item A
3-4	price1	00 50	$0x0050 = 5 \times 16 = 80$
5	item2	22	Item B
6-7	price2	00 C8	$0x00C8 = 12 \times 16 + 8 = 200$

64-bit response (FPGA to judge): a possible response is 00 10 11 01 22 02 00 00

Bytes	Field	Hex	Meaning
0-1	index	00 10	16, echoed
2	item1	11	Item A, echoed
3	action1	01	SELL
4	item2	22	Item B, echoed
5	action2	02	BUY
6-7	reserved	00 00	must stay zero

11. Scoring

Each team is scored out of 100: correctness 70 points, latency 15, and LUT usage 15.

Criterion	Points	Measured from	How points are awarded
Packet correctness	50	Packets with the right index, both item IDs, and both actions, out of 84 scored packets	$50 \times \text{correct packets} \div 84$
Action correctness	20	Individual actions, out of 168 scored actions	$20 \times \text{correct actions} \div 168$

Latency	15	Average round-trip latency of received packets, compared with the organizer reference design on the same judge PC	15 if at most 1.25× the reference average; 8 if at most 2×; otherwise 0. Zero if packet correctness is below 95%
LUT usage	15	Total LUT count in the Gowin synthesis report, compared with the reference design	15 × min(1, reference LUTs ÷ your LUTs). Zero if packet correctness is below 95%
TOTAL	100		

Reference values: average latency 16.626 ms and 542 LUTs (the average of the VHDL and Verilog reference designs, from the LUT line of the Gowin Resource Usage Summary). The latency tiers therefore work out to about 20.8 ms (1.25×) and 33.3 ms (2×).

Latency. Latency is measured from just before the judge transmits a request until the complete 8-byte response has been received. It includes UART transfer in both directions, your FPGA's processing, and USB/serial overhead on the PC. At 115200 baud (8N1) the 16 bytes of one transaction need only about 1.39 ms of wire time, so the roughly 16.6 ms measured on the organizers' test PC is dominated by BL616, USB, operating-system, and serial-buffering overhead rather than by the UART itself or your logic. Any idle time your design adds between response bytes is included in the measurement. Expect only small differences between correct designs.

Reading your LUT count. After synthesis, double-click Synthesis Report in the Process pane, open Resource → Resource Usage Summary, and record the total LUT usage. Gowin also shows a breakdown (LUT2, LUT3, LUT4, and so on). The total is the number used for judging.

12. Troubleshooting

UART and the COM port. The PC talks to the FPGA through the Tang Nano 20K's onboard BL616 chip, which appears as a USB serial (COM) port. Find the port number in Windows Device Manager and set PORT in the Python test scripts to match. Only one program can open the port at a time, so close serial terminals and other scripts before running a test.

Timeouts or corrupted responses although your logic is correct. The BL616 can drop or corrupt bytes if your design transmits response bytes with no idle time between them. Add idle time or buffering between bytes, rerun the robust test, and check the CSV for which packets failed.

"Cable failed to open via the location" or "No Gowin devices found." Try, in order:

1. Close Python, serial terminals, and other Programmer instances.
2. Reconnect the board directly to the PC rather than through a hub.
3. Use a known data-capable USB-C cable.
4. Retry Scan Device.
5. Try another detected USB Debugger A/... location.
6. Use the latest standalone Gowin Programmer.
7. Use openFPGALoader as a fallback.
8. Ask an organizer before changing Windows USB drivers or BL616 firmware.

Sipeed specifically recommends the latest Programmer when no Gowin device is found, and documents debugger-firmware updating as a later troubleshooting option.

13. Support documentation

1. Sipeed Tang Nano 20K (contains schematic and datasheet)
2. Sipeed Nano 20K Gowin tutorial
3. Sipeed Education Edition installation
4. Sipeed Tang troubleshooting
5. Sipeed debugger-firmware instructions
6. Gowin Programmer User Guide
7. openFPGALoader first steps
8. msys2.org
9. openFPGALoader supported boards
10. Intel FPGA software/Quarta installation support
11. Questa-Intel FPGA Edition Quick Start
12. GQH instructions repository: github.com/ShayanNazir/GQH-Hardware-Track-Submission

Part 3: Final Submission Instructions

The official submission channel is Devpost. Each team submits its own project repository through Devpost. Do not upload team project code into the GQH instructions repository.

Official links & dates

Devpost submission URL	https://gqhacks.devpost.com
Submission deadline	Sunday, October 4, 11:00 am EDT
Questions / support	Use the official GQH Hardware Track communication channel (Discord: discord.gg/9qPMtN4UB)

1. Before you submit

- Each team should maintain its own GitHub repository for its project.
- Use the GQH instructions repository only for submission requirements, repository structure guidance, the team README template, and the submission checklist.
- Make sure the repository represents the same project and version you hand in at drop-off.

2. Prepare your final GitHub repository

Your repository should contain the complete version of the project that your team wants judged. At minimum, include:

- Final HDL source files used by the FPGA design
- Tang Nano 20K physical constraint file(s), including the organizer-supplied .cst file
- Gowin project/build files needed to reproduce the design
- The generated .fs programming file, built from that exact source (judges program the board from it)

- A complete README .md
- Any host-side test or demo code (it is not executed during judging)
- Testbench/simulation files, if used
- Supporting data/configuration files needed to reproduce the project

Recommended repository layout.

```
team-project/
|-- README.md
|-- src/           HDL source files
|-- constraints/   19_tang_nano_20k.cst
|-- testbench/     testbench files
|-- gowin/         Gowin project/build files
|-- bitstream/     final .fs file
`-- results/       optional test CSVs and benchmarks
```

3. README requirements

Your project README should let a judge understand and reproduce the project without guessing. Include:

- Team/project name and team members
- Brief project description
- What functionality is implemented directly on the FPGA
- Any host-side tooling you used for local testing (host software is not run during judging)
- FPGA board: Tang Nano 20K
- HDL/languages used
- Gowin EDA version used
- Top-level entity/module name
- Build instructions and FPGA programming instructions
- Project inputs and expected outputs; how to reproduce the final demo
- Testing/verification procedure
- Relevant performance results, including your LUT count
- External libraries, IP cores, starter code, datasets, or other pre-existing resources used
- Known limitations or incomplete features

Organizer setup reference: the FPGA guide in Part 2 uses Gowin EDA V1.9.11.03 Education and targets GW2AR-LV18QN88C8/I7. Document your actual final configuration clearly.

4. Verify the final FPGA build

Before freezing your submission, verify that the repository corresponds to a working build. Confirm that:

- ☐ The project synthesizes successfully
- ☐ Place & Route completes successfully
- ☐ The correct Tang Nano 20K .cst constraint file is included
- ☐ Top-level port names match the .cst constraints
- ☐ The design can be programmed onto the Tang Nano 20K

- ☐ The quick and robust UART tests pass on your board

5. Freeze your final GitHub version

Once your team is satisfied with the final version, commit and push all final changes:

```
git status
git add .
git commit -m "Final hackathon submission"
git push
git rev-parse HEAD
```

Save the full Git commit SHA. This value identifies the exact version of the project being submitted for judging. Do not rely only on the repository URL, because the repository can continue changing after the deadline.

6. Make sure judges can access the repository

- **Public repository:** verify that the repository link works when you are logged out of GitHub.
- **Private repository:** follow the organizer-provided judging-access instructions and grant access before the submission deadline.

Do not commit secrets. Never place passwords, API keys, access tokens, private keys, or personal credentials in the repository.

7. Submit through Devpost

Devpost is the official submission portal for the GQH Hardware Track. For the hardware/code portion, make sure your submission clearly includes or references:

- GitHub repository URL
- Full final Git commit SHA
- Demo video, if required
- Project description and any other fields required by the official Devpost submission

If Devpost does not provide a dedicated field for the final commit SHA, place it clearly in your Devpost project description and in your GitHub README, for example:

```
Final GitHub Submission
Repository: https://github.com/team-name/project-name
Final Commit SHA: 7fe929310cd84d0e1f1d6c1234567890abcdef12
```

8. Final submission checklist

- ☐ Final source code is pushed to GitHub
- ☐ Repository is accessible to judges
- ☐ Tang Nano 20K .cst file is included
- ☐ Gowin project/build files and the final .fs file are included
- ☐ README is complete
- ☐ Top-level entity/module is documented
- ☐ Build and FPGA programming instructions are included
- ☐ Project successfully synthesizes

- ☐ Place & Route successfully completes
- ☐ Final design has been tested on the Tang Nano 20K
- ☐ External resources are disclosed
- ☐ Final Git commit has been pushed
- ☐ Full final commit SHA has been recorded
- ☐ GitHub repository URL is included in Devpost
- ☐ Final commit SHA is included in Devpost/project documentation
- ☐ Demo video is included if required
- ☐ Devpost submission is complete before 11:00 am EDT on October 4
- ☐ Board and accessories are dropped off at Reitz Room 2345 by 11:00 am on October 4

Submission freeze: judging is based on the final Git commit SHA identified in the team's submission. Changes pushed to the repository after the deadline may not be considered part of the judged submission.